Generazione e valutazione di grafi mediante stima e diffusione di graphon

TODO: Nome Cognome

18 luglio 2026

Indice

1 Introduzione **3**
- 1.1 Struttura della pipeline e contributi . . . 4
- 1.2 Obiettivi . . . 4
- 1.3 Fondamenti bibliografici . . . 5

2 Contesto teorico **7**
- 2.1 Graphon e generazione di grafi . . . 7
  - 2.1.1 Non identificabilita' e problema dell'ordinamento . . . 8
  - 2.1.2 Discretizzazione . . . 8
- 2.2 Stima del graphon: SIGL . . . 9
- 2.3 Diffusione in spazi discreti: il framework D3PM . . . 10
  - 2.3.1 Processo diretto . . . 10
  - 2.3.2 Scelta della matrice di transizione . . . 11
  - 2.3.3 Schedule di rumore . . . 11
  - 2.3.4 Parametrizzazione inversa e loss . . . 12
- 2.4 Diffusione discreta su grafi: DiGress . . . 12
  - 2.4.1 Processo diretto fattorizzato . . . 13
  - 2.4.2 Rete di denoising e loss . . . 13
  - 2.4.3 Proprieta' di equivarianza . . . 13
  - 2.4.4 Feature ausiliarie strutturali . . . 14
- 2.5 Valutazione tramite MMD e il protocollo GGME . . . 14
  - 2.5.1 Desiderata . . . 14
  - 2.5.2 Descrittori e MMD . . . 15
  - 2.5.3 Selezione di kernel e iperparametri . . . 15

3 La pipeline OGAD **17**
- 3.1 Dataset . . . 17
- 3.2 Stadio 1: stima dei graphon con SIGL . . . 18
  - 3.2.1 Il problema della cardinalita' del training set . . . 18
  - 3.2.2 Iperparametri e architettura effettivi . . . 19
  - 3.2.3 Formato di uscita . . . 20

3.3 Stadio 2, variante A: OGAD–D3PM 20
3.3.1 Rappresentazione del dato 20
3.3.2 Processo diffusivo 21
3.3.3 Rete di denoising 21
3.3.4 Addestramento 22
3.3.5 Generazione 23
3.4 Stadio 2, variante B: OGAD–DiGress 23
3.4.1 Riduzione del graphon a un grafo etichettato 23
3.4.2 Processo diffusivo 24
3.4.3 Rete di denoising e loss 24
3.4.4 Addestramento e generazione 25
3.5 Confronto tra le due varianti 26
3.6 Stadio 3: campionamento dei grafi 27
3.7 Stadio 4: valutazione GGME 28
3.7.1 Descrittori 28
3.7.2 Protocollo di selezione 28
3.7.3 Selezione del candidato D3PM tramite MMD 29
3.7.4 Termini di confronto 30
3.8 Orchestrazione e riproducibilita' 30

4 Risultati preliminari **32**
4.1 Risultati riportati 32
4.2 Lettura dei risultati 32
4.3 Ipotesi sulle sorgenti di errore 33

5 Conclusioni provvisorie **35**

Capitolo 1

Introduzione

La generazione di grafi e' un problema centrale nello studio di dati relazionali: molte applicazioni, dalle reti sociali ai sistemi biologici, richiedono modelli in grado di produrre nuove istanze che preservino proprieta' strutturali osservate in un insieme di esempi. A differenza dei dati vettoriali o delle immagini, i grafi combinano una struttura combinatoria discreta con vincoli di invarianza rispetto alla permutazione dei nodi. Questa natura rende non banale sia la definizione di modelli generativi efficaci, sia la loro valutazione quantitativa.

Il presente lavoro considera una pipeline sperimentale per la generazione di grafi che introduce una *rappresentazione intermedia* basata su graphon. Un graphon puo' essere interpretato come una funzione simmetrica $W : [0, 1]^2 \to [0, 1]$, dalla quale e' possibile campionare grafi scegliendo variabili latenti associate ai nodi e generando archi indipendenti con probabilita' data dal valore di W. In questa prospettiva, il graphon fornisce una rappresentazione continua e non parametrica della struttura probabilistica di una famiglia di grafi.

L'idea alla base della pipeline studiata in questa tesi, indicata nel repository come **OGAD**, e' che il modello generativo profondo non debba apprendere direttamente una distribuzione su matrici di adiacenza, ma una distribuzione *sui graphon*. Il vantaggio atteso e' duplice. In primo luogo, la rappresentazione a graphon e' invariante rispetto alla dimensione del grafo: un unico oggetto 128×128 puo' generare grafi di qualunque numero di nodi. In secondo luogo, la rappresentazione e' gia' allineata (i nodi sono ordinati in base alle variabili latenti stimate), il che elimina il problema della permutazione dal modello generativo e lo riduce a un problema di modellazione di un tensore denso a valori in $[0, 1]$.

1.1 Struttura della pipeline e contributi

La pipeline si articola in quattro stadi, illustrati in dettaglio nel Capitolo 3:

1. **Stima dei graphon.** A partire da un insieme di grafi osservati, SIGL (*Scalable Implicit Graphon Learning*) stima graphon mediante rappresentazioni neurali implicite (INR) e reti neurali su grafi [2]. Una modifica introdotta in questo lavoro consente di ottenere da un singolo dataset non un unico graphon ma un intero *campione* di graphon, tramite clustering dei grafi seguito da bootstrap non parametrico.

2. **Modellazione generativa dei graphon.** I graphon stimati sono usati come dataset di training per un modello di diffusione in spazio discreto. Sono implementate due varianti alternative e mutuamente esclusive:

   - **OGAD–D3PM**, che tratta il graphon quantizzato come un'immagine categoriale 128×128 e applica il framework D3PM [1] con un denoiser di tipo Diffusion Transformer [18];
   - **OGAD–DiGress**, che tratta il graphon quantizzato come un *grafo completo* su 128 nodi con etichette d'arco categoriali, e applica il modello DiGress [25] senza modifiche al nucleo diffusivo.

3. **Campionamento dei grafi.** I graphon generati vengono sanificati e usati come kernel di Bernoulli per campionare grafi, con numero di nodi estratto dalla distribuzione empirica del test set.

4. **Valutazione.** I grafi generati sono confrontati con il test set tramite MMD su descrittori strutturali, seguendo il protocollo di selezione di kernel e iperparametri proposto da GGME [15].

Il contributo principale di questa tesi non e' la proposta di un nuovo modello diffusivo, ma la *costruzione e la caratterizzazione precisa* di una pipeline composita che collega quattro lavori indipendenti (SIGL, D3PM, DiGress, GGME) in un protocollo sperimentale riproducibile, insieme all'analisi delle scelte di progetto — in particolare della discretizzazione dei valori del graphon — che determinano la qualita' del risultato finale.

1.2 Obiettivi

Gli obiettivi sperimentali possono essere riassunti come segue:

- stimare un insieme di graphon rappresentativi del training set mediante SIGL, con una procedura di clustering e bootstrap che ne aumenti la cardinalita' fino a rendere possibile l'addestramento di un modello generativo;
- addestrare un generatore diffusivo discreto sui graphon stimati, nelle due varianti D3PM e DiGress;
- confrontare la pipeline OGAD con due riferimenti: DiGress applicato direttamente ai grafi (baseline), e i grafi indicati come «Paper», ossia le predizioni di esempio distribuite con il repository GGME;
- valutare i grafi generati tramite MMD su distribuzione dei gradi, coefficiente di clustering e spettro del laplaciano normalizzato.

1.3 Fondamenti bibliografici

Il lavoro si colloca all'intersezione tra teoria dei graphon, stima statistica di reti e modelli generativi profondi per grafi. Dal punto di vista teorico, i graphon nascono come oggetti limite per sequenze dense di grafi [13, 12]; la connessione con i modelli di grafo scambiabili e con la rappresentazione di Aldous–Hoover e' discussa in modo esplicito da Diaconis e Janson [6]. Questa base giustifica l'uso del graphon come rappresentazione probabilistica non parametrica della struttura di una famiglia di grafi.

Sul piano statistico, la stima di graphon e' stata studiata sia come problema minimax sia come problema computazionale. Gao, Lu e Zhou stabiliscono tassi ottimali per la stima di graphon e mettono in evidenza il legame con modelli a blocchi stocastici e regressione non parametrica [8]. Borgs e Chayes presentano invece una prospettiva piu' ampia sull'uso dei graphon per modellare, stimare e progettare algoritmi su reti massive [3]. Chan e Airoldi introducono lo stimatore a istogramma consistente su grafi ordinati [4] che costituisce il secondo stadio di SIGL, mentre IGNR [26] e SIGL [2] sostituiscono la matrice a risoluzione fissa con una rappresentazione neurale implicita continua.

Per quanto riguarda la generazione di grafi, GraphRNN rappresenta un riferimento classico per i modelli autoregressivi e ha contribuito a rendere comune la valutazione tramite MMD su descrittori strutturali [27]. Altri lavori, come GraphVAE e MolGAN, mostrano approcci alternativi basati rispettivamente su autoencoder variazionali e modelli avversari [19, 5]. La componente diffusiva della tesi si collega ai diffusion probabilistic models [21, 10], alla formulazione score-based tramite equazioni differenziali stocastiche

[22], e alla loro estensione a spazi discreti tramite multinomial diffusion [11] e D3PM [1], di cui DiGress [25] costituisce la specializzazione al dominio dei grafi.

Capitolo 2

Contesto teorico

Questo capitolo raccoglie la formalizzazione dei quattro componenti della pipeline. L'obiettivo e' fissare la notazione che verra' usata nel Capitolo 3, dove ciascun componente viene specializzato al caso concreto implementato nel repository.

2.1 Graphon e generazione di grafi

Definizione 2.1 (Graphon)**.** Un graphon e' una funzione misurabile, simmetrica e limitata

$$W : [0,1]^2 \to [0,1], \qquad W(u,v) = W(v,u) \quad \text{per quasi ogni } (u,v).$$

Dato un intero $M \in \mathbb{N}$, il graphon induce una distribuzione su grafi semplici non orientati con M nodi. Il modello generativo associato, detto *W-random graph*, e'

$$\begin{aligned} \eta_1, \dots, \eta_M &\overset{\text{i.i.d.}}{\sim} \text{Uniform}([0,1]), \\ A_{ij} \mid \eta_i, \eta_j, W &\sim \text{Bernoulli}\big(W(\eta_i, \eta_j)\big), \qquad 1 \le i < j \le M, \end{aligned} \tag{2.1}$$

con $A_{ji} = A_{ij}$ e $A_{ii} = 0$. Condizionatamente alle variabili latenti dei nodi, gli archi sono indipendenti e la probabilita' di un arco dipende solo dal valore del graphon nella coppia di coordinate latenti:

$$\mathbb{P}(A_{ij} = 1 \mid \eta_i, \eta_j, W) = W(\eta_i, \eta_j).$$

Il teorema di Aldous–Hoover garantisce che ogni distribuzione scambiabile su grafi infiniti ammette una rappresentazione di questa forma [6], il che giustifica l'uso del graphon come modello non parametrico universale per famiglie di grafi scambiabili.

2.1.1 Non identificabilita' e problema dell'ordinamento

La rappresentazione (2.1) non e' identificabile. Se $\varphi : [0,1] \to [0,1]$ e' una trasformazione misura-preservante, allora

$$W'(u,v) = W(\varphi(u), \varphi(v)) \quad \text{q.o.} \tag{2.2}$$

induce esattamente la stessa distribuzione su grafi. Ne segue che la classe di equivalenza $[W]$, e non W stesso, e' l'oggetto identificabile.

Questo fatto ha due conseguenze operative centrali per la pipeline OGAD:

1. **Lato stima.** Uno stimatore di graphon deve fissare un rappresentante della classe, cioe' scegliere un *ordinamento* dei nodi. I metodi classici (SAS, USVT) ordinano per grado empirico [4, 2]; SIGL apprende invece l'ordinamento tramite una GNN, il che gli consente di trattare anche graphon non monotoni come i modelli a blocchi stocastici, dove tutti i nodi hanno lo stesso grado atteso.

2. **Lato generazione.** Poiche' il modello diffusivo apprende una distribuzione su *matrici* 128×128 e non su classi di equivalenza, la coerenza dell'ordinamento prodotto da SIGL sui diversi graphon del training set e' un prerequisito: se due graphon equivalenti fossero rappresentati con permutazioni latenti diverse, il modello dovrebbe spendere capacita' per apprendere l'orbita della permutazione anziche' la struttura.

2.1.2 Discretizzazione

Nell'implementazione, W non e' rappresentato come funzione continua ma come matrice $R \times R$, con $R = 128$. Indicando con $\widehat{W} \in [0,1]^{R \times R}$ tale discretizzazione, il campionamento di (2.1) viene approssimato scegliendo indici

$$z_i = \lfloor R\,\eta_i \rfloor \wedge (R-1), \qquad z_i \in \{0, \dots, R-1\}, \tag{2.3}$$

e ponendo

$$A_{ij} \mid z_i, z_j, \widehat{W} \sim \text{Bernoulli}\big(\widehat{W}_{z_i z_j}\big).$$

Prima del campionamento si applica una proiezione $\Pi : \mathbb{R}^{R \times R} \to [0,1]^{R \times R}$ che impone i vincoli strutturali di grafo semplice non orientato:

$$\Pi(X)_{ij} = \begin{cases} 0, & i = j, \\ \min\big\{1, \max\{0, (X_{ij} + X_{ji})/2\}\big\}, & i \neq j. \end{cases} \tag{2.4}$$

Ossia: simmetrizzazione, clipping in $[0,1]$, diagonale nulla. Questa proiezione ricorre in tre punti distinti della pipeline (uscita di SIGL, uscita del generatore, ingresso del campionatore) ed e' l'unico meccanismo che garantisce la

validita' strutturale del graphon, dal momento che nessuno dei due modelli generativi impone la simmetria a livello architetturale.

2.2 Stima del graphon: SIGL

SIGL [2] risolve il seguente problema: dato un insieme di grafi $\mathcal{D} = \{A_t\}_{t=1}^{M}$ generati da un graphon incognito ω, stimare ω. Rispetto a IGNR [26], che minimizza la distanza di Gromov–Wasserstein e ha complessita' cubica nel numero di nodi, SIGL sostituisce la GW con un MSE puntuale, il che richiede pero' di risolvere esplicitamente il problema di allineamento. La procedura si articola in tre passi.

Passo 1: apprendimento dell'ordinamento. Si stimano le variabili latenti $\hat{\eta}_t = \{\hat{\eta}_i\}_{i=1}^{n_t}$ di ogni grafo mediante una GNN g_{ϕ_1}, e simultaneamente un *graphon ausiliario* h_{ϕ_2}, rappresentato da una INR, che funge da riferimento comune per tutti i grafi:

$$\mathcal{L}(\phi) = \sum_{t=1}^{M} \frac{1}{n_t^2} \sum_{i,j=1}^{n_t} \big[A_t(i,j) - h_{\phi_2}\big(\hat{\eta}_t(i), \hat{\eta}_t(j)\big)\big]^2, \qquad \hat{\eta}_t = g_{\phi_1}(A_t, X_t). \tag{2.5}$$

Le feature di nodo X_t sono rumore gaussiano, cosicche' l'informazione usata per l'ordinamento e' interamente strutturale. Si definisce quindi la permutazione $\hat{\pi}$ che ordina i nodi per valore decrescente di $\hat{\eta}$, e si ottiene la matrice di adiacenza ordinata $\hat{A}_t(i,j) = A_t(\hat{\pi}(i), \hat{\pi}(j))$.

Passo 2: approssimazione a istogramma. Seguendo Chan e Airoldi [4], si applica un average pooling non sovrapposto di finestra h alla matrice ordinata:

$$\hat{H}_t(i,j) = \frac{1}{h^2} \sum_{s_1=1}^{h} \sum_{s_2=1}^{h} \hat{A}_t\big((i-1)h + s_1,\ (j-1)h + s_2\big), \qquad \hat{H}_t \in [0,1]^{k_t \times k_t}, \tag{2.6}$$

con $k_t = \lfloor n_t/h \rfloor$. Ciascun $\hat{H}_t$ e' una realizzazione rumorosa del graphon vero, osservata a una risoluzione diversa.

Passo 3: regressione INR. Si costruisce il dataset supervisionato

$$\mathcal{C} = \Big\{ \big(\tfrac{i}{k_t}, \tfrac{j}{k_t}, \hat{H}_t(i,j)\big) \ :\ i,j \in \{1,\dots,k_t\},\ t \in \{1,\dots,M\} \Big\},$$

e si addestra una INR $f_\theta : [0,1]^2 \to [0,1]$ minimizzando un MSE pesato:

$$\theta^\star = \arg\min_\theta \sum_{t=1}^{M} w_t \sum_{i=1}^{k_t}\sum_{j=1}^{k_t} \Big(\hat{H}_t(i,j) - f_\theta\Big(\tfrac{i}{k_t}, \tfrac{j}{k_t}\Big)\Big)^2, \qquad w_t = \frac{|G_t|}{\sum_{i=1}^{M}|G_i|}. \tag{2.7}$$

Il peso w_t proporzionale al numero di nodi privilegia i grafi piu' grandi, la cui risoluzione fornisce una stima migliore del graphon vero. La INR e' una rete SIREN [20], cioe' un MLP con attivazioni sinusoidali $\phi_i(x) = \sin(\omega_0 W_i x + b_i)$, scelta per la capacita' di rappresentare segnali ad alta frequenza. Essendo f_θ definita sull'intero dominio continuo, il graphon stimato e' *resolution-free*: puo' essere campionato a qualunque risoluzione R.

La Proposizione 1 di SIGL stabilisce che, se ω e' Lipschitz con costante $\mathcal{L}$ e si sceglie $h = \log n$, allora

$$\lim_{n\to\infty} \mathbb{E}\big[\|f_\theta - \omega\|_2^2\big] \le \varepsilon_{\mathrm{tr}} + 4\mathcal{L}^2\tau^2, \tag{2.8}$$

dove $\varepsilon_{\mathrm{tr}}$ e' l'errore massimo di regressione della INR sui punti di $\mathcal{C}$ e τ e' l'errore massimo della GNN nella stima delle variabili latenti. L'errore asintotico e' quindi controllato *solo* dalla capacita' espressiva dei due moduli: architetture piu' espressive producono stimatori piu' consistenti.

2.3 Diffusione in spazi discreti: il framework D3PM

I modelli di diffusione apprendono a invertire un processo di corruzione progressiva dei dati [21, 10]. Nei modelli continui la corruzione e' aggiunta di rumore gaussiano; in spazi discreti occorre invece definire transizioni tra stati categoriali. D3PM [1] fornisce il framework generale.

2.3.1 Processo diretto

Sia $x_0 \in \{1,\dots,K\}$ una variabile categoriale e $\boldsymbol{x}_0$ la sua codifica one-hot come vettore riga. Il processo diretto e' una catena di Markov

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \qquad q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\, \boldsymbol{x}_{t-1} Q_t\big), \tag{2.9}$$

dove $[Q_t]_{ij} = q(x_t = j \mid x_{t-1} = i)$ e' una matrice stocastica per riga. Poiche' la catena e' markoviana, la marginale a t passi e la posterior sono disponibili

in forma chiusa:

$$q(x_t \mid x_0) = \mathrm{Cat}\big(x_t;\, \boldsymbol{x}_0 \overline{Q}_t\big), \qquad \overline{Q}_t = Q_1 Q_2 \cdots Q_t, \tag{2.10}$$

$$q(x_{t-1} \mid x_t, x_0) = \mathrm{Cat}\left(x_{t-1};\, \frac{\boldsymbol{x}_t Q_t^\top \odot \boldsymbol{x}_0 \overline{Q}_{t-1}}{\boldsymbol{x}_0 \overline{Q}_t \boldsymbol{x}_t^\top}\right), \tag{2.11}$$

con $\odot$ prodotto puntuale. Sono le due proprieta' che rendono possibile l'addestramento efficiente: (2.10) consente di campionare direttamente x_t da x_0 per un t arbitrario (training parallelo sui timestep), (2.11) consente di calcolare analiticamente le divergenze KL della loss.

2.3.2 Scelta della matrice di transizione

D3PM mostra che la scelta di Q_t e' una decisione di progetto rilevante. Le due varianti pertinenti a questo lavoro sono:

- **Uniforme** [11]:

$$Q_t = (1-\beta_t) I + \frac{\beta_t}{K}\, \mathbf{1}\mathbf{1}^\top, \qquad [Q_t]_{ij} = \begin{cases} 1 - \frac{K-1}{K}\beta_t, & i = j, \\ \frac{\beta_t}{K}, & i \neq j. \end{cases} \tag{2.12}$$

  La matrice e' doppiamente stocastica a entrate strettamente positive, quindi per il teorema di Perron–Frobenius la distribuzione stazionaria e' uniforme su $\{1, \ldots, K\}$. E' questa la variante usata in *entrambe* le configurazioni OGAD.

- **Marginale** [25]: $Q_t = \alpha^t I + \beta^t \mathbf{1}\boldsymbol{m}^\top$, con $\boldsymbol{m}$ la distribuzione marginale empirica delle classi nel training set. La distribuzione limite e' $\boldsymbol{m}$ anziche' uniforme; l'idea e' che una prior piu' vicina alla distribuzione dei dati renda il training piu' facile. Il Teorema 4.1 di DiGress mostra che il prodotto delle marginali e' la proiezione ortogonale della distribuzione vera sulla classe delle distribuzioni prodotto.

2.3.3 Schedule di rumore

Lo schedule determina β_t. La pipeline usa in entrambe le varianti uno schedule di tipo coseno, ispirato a Nichol e Dhariwal [14], che fissa la probabilita' cumulata di non-transizione $\bar{\alpha}_t$ e ne deduce β_t:

$$\bar{\alpha}(s) = \cos\left(\frac{s + s_0}{1 + s_0} \cdot \frac{\pi}{2}\right)^p, \qquad \beta_t = 1 - \frac{\bar{\alpha}(t/T)}{\bar{\alpha}((t-1)/T)}, \qquad s_0 = 0.008. \tag{2.13}$$

Le due implementazioni differiscono nell'esponente p (Sezione 3.3 e Sezione 3.4): D3PM nel repository usa $p = 1$, DiGress usa $p = 2$ come nella formulazione originale.

2.3.4 Parametrizzazione inversa e loss

Invece di predire direttamente i logit di $p_\theta(x_{t-1} \mid x_t)$, si segue la x_0-parametrizzazione: una rete nn_θ predice i logit di $\tilde{p}_\theta(\tilde{x}_0 \mid x_t)$, che vengono combinati con la posterior esatta:

$$p_\theta(x_{t-1} \mid x_t) \;\propto\; \sum_{\tilde{x}_0} q(x_{t-1}, x_t \mid \tilde{x}_0)\, \tilde{p}_\theta(\tilde{x}_0 \mid x_t). \tag{2.14}$$

Questa scelta ha tre vantaggi: (i) la KL in (2.15) si annulla esattamente quando $\tilde{p}_\theta$ concentra tutta la massa su x_0; (ii) il pattern di sparsita' di Q_t e' automaticamente ereditato da p_θ; (iii) consente inferenza a k passi alla volta, $p_\theta(x_{t-k} \mid x_t) = \sum q(x_{t-k}, x_t \mid \tilde{x}_0)\tilde{p}_\theta(\tilde{x}_0 \mid x_t)$.

L'addestramento minimizza un bound variazionale sulla negative log-likelihood:

$$L_{\mathrm{vb}} = \mathbb{E}_{q(x_0)}\Bigg[\underbrace{D_{\mathrm{KL}}\big(q(x_T \mid x_0) \,\|\, p(x_T)\big)}_{L_T} + \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(x_t|x_0)} D_{\mathrm{KL}}\big(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)\big)}_{L_{t-1}} \underbrace{- \mathbb{E}_{q(x_1|x_0)}}$$

(2.15)

Il termine L_T non dipende da θ se la prior e' fissata alla distribuzione stazionaria del processo diretto. D3PM introduce inoltre una loss ibrida che aggiunge un termine ausiliario di cross-entropy sulla predizione di x_0:

$$L_\lambda = L_{\mathrm{vb}} + \lambda\, \mathbb{E}_{q(x_0)} \mathbb{E}_{q(x_t|x_0)} \big[-\log \tilde{p}_\theta(x_0 \mid x_t)\big]. \tag{2.16}$$

Il termine ausiliario coincide con L_0 per $t = 1$ ed e' minimizzato dalla stessa configurazione che minimizza L_{vb}; la sua funzione e' stabilizzare il gradiente, la cui varianza in L_{vb} dipende fortemente da t. Nel repository λ e' l'iperparametro `hybrid_loss_coeff`.

Nel caso multivariato — che e' il caso di interesse, dato che un graphon ha R^2 celle — si assume che Q_t sia applicata indipendentemente a ogni componente e che sia q sia p_θ fattorizzino sulle componenti. Le KL si calcolano dunque componente per componente e si sommano.

2.4 Diffusione discreta su grafi: DiGress

DiGress [25] specializza il framework precedente al caso in cui il dato e' un grafo con attributi categoriali su nodi e archi. Un grafo con n nodi e'

rappresentato dalla coppia $G = (X, E)$, con $X \in \mathbb{R}^{n \times a}$ matrice delle codifiche one-hot dei tipi di nodo (a classi) ed $E \in \mathbb{R}^{n \times n \times b}$ tensore delle codifiche one-hot dei tipi di arco (b classi, dove l'assenza di arco e' trattata come un tipo particolare).

2.4.1 Processo diretto fattorizzato

Il rumore viene applicato indipendentemente a ogni nodo e a ogni arco, tramite due matrici di transizione distinte $Q_X^t \in \mathbb{R}^{a \times a}$ e $Q_E^t \in \mathbb{R}^{b \times b}$:

$$q(G^t \mid G^{t-1}) = \big(X^{t-1}Q_X^t,\ E^{t-1}Q_E^t\big), \qquad q(G^t \mid G) = \big(X\overline{Q}_X^t,\ E\overline{Q}_E^t\big). \tag{2.17}$$

Nel caso di grafi non orientati il rumore e' applicato solo alla parte triangolare superiore di E, che viene poi simmetrizzata. Si noti che lo spazio degli stati su cui e' definita la matrice di transizione non e' lo spazio dei grafi (che sarebbe di cardinalita' proibitiva) ma solo l'insieme dei tipi di nodo e di arco.

2.4.2 Rete di denoising e loss

La rete φ_θ riceve G^t e predice una distribuzione $\hat{p}^G = (\hat{p}^X, \hat{p}^E)$ sul grafo pulito. L'addestramento minimizza una cross-entropy che si decompone su nodi e archi:

$$l(\hat{p}^G, G) = \sum_{1 \le i \le n} \mathrm{CE}\big(x_i, \hat{p}_i^X\big) \ + \ \lambda \sum_{1 \le i,j \le n} \mathrm{CE}\big(e_{ij}, \hat{p}_{ij}^E\big), \tag{2.18}$$

con $\lambda \in \mathbb{R}^+$ che bilancia il peso relativo di nodi e archi. Il campionamento inverso marginalizza sulle predizioni della rete:

$$p_\theta(G^{t-1} \mid G^t) = \prod_i p_\theta(x_i^{t-1} \mid G^t) \prod_{i,j} p_\theta(e_{ij}^{t-1} \mid G^t), \tag{2.19}$$

$$p_\theta(x_i^{t-1} \mid G^t) = \sum_{x \in \mathcal{X}} q\big(x_i^{t-1} \mid x_i = x,\, x_i^t\big)\, \hat{p}_i^X(x), \tag{2.20}$$

e analogamente per gli archi.

2.4.3 Proprieta' di equivarianza

Tre lemmi di DiGress garantiscono la correttezza rispetto alle permutazioni: l'architettura e' equivariante (Lemma 3.1), la loss (2.18) e' invariante perche' decomponibile per nodo e per arco (Lemma 3.2), e la distribuzione generata e' scambiabile (Lemma 3.3), poiche' la prior e' un prodotto di distribuzioni

i.i.d. e le transizioni sono equivarianti. In particolare il Lemma 3.2 implica che non e' necessario alcun graph matching tra grafo predetto e grafo target, a differenza degli approcci basati su VAE.

Osservazione 2.2. Nella variante OGAD–DiGress (Sezione 3.4) queste proprieta' restano valide ma perdono rilevanza pratica: il «grafo» in ingresso e' un grafo completo su 128 nodi con tipi di nodo costanti, quindi l'unica informazione risiede nelle etichette d'arco e la permutazione dei nodi corrisponde proprio alla riparametrizzazione (2.2) che SIGL ha gia' fissato. L'equivarianza della rete diventa quindi un vincolo che il modello deve «superare»: la struttura del graphon e' *ordinata*, e un modello equivariante non puo' sfruttare direttamente tale ordinamento se non attraverso i valori delle etichette.

2.4.4 Feature ausiliarie strutturali

Poiche' le MPNN standard hanno potere espressivo limitato (non sanno contare i cicli), DiGress aumenta l'input della rete con descrittori calcolati sul grafo rumoroso: conteggi di cicli fino a lunghezza 6, numero di componenti connesse, primi autovalori del laplaciano e relativi autovettori. Questa possibilita' e' resa disponibile proprio dalla natura discreta del rumore, che preserva la sparsita' del grafo rumoroso (a differenza della diffusione gaussiana, che produce grafi completi in cui le nozioni topologiche non sono definite).

2.5 Valutazione tramite MMD e il protocollo GGME

La valutazione dei modelli generativi per grafi e' particolarmente delicata: diversamente dalle immagini, i grafi non possono essere valutati per ispezione visiva, e le metriche comunemente adottate risultano instabili se non associate a descrittori strutturali e kernel appropriati. GGME [15] affronta sistematicamente questo problema.

2.5.1 Desiderata

GGME enuncia tre requisiti per una metrica di confronto tra distribuzioni di grafi:

- **Espressivita'**: $d(\mathcal{G}, \mathcal{G}')$ deve crescere monotonicamente al crescere della dissimilarita';

- **Robustezza**: piccole perturbazioni della distribuzione devono produrre piccole variazioni della metrica;
- **Efficienza**: il costo computazionale deve scalare con il numero e la dimensione dei grafi.

2.5.2 Descrittori e MMD

Siano $\mathcal{P}$ e $\mathcal{Q}$ due distribuzioni su grafi e sia $\phi : \mathcal{G} \to \mathbb{R}^m$ un descrittore strutturale. Data una funzione kernel positiva definita $k : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}$, la MMD tra le distribuzioni indotte dai descrittori e'

$$\begin{aligned}\mathrm{MMD}_k^2(\mathcal{P}, \mathcal{Q}) = \mathbb{E}_{G,G' \sim \mathcal{P}}\, k(\phi(G), \phi(G')) + \mathbb{E}_{H,H' \sim \mathcal{Q}}\, k(\phi(H), \phi(H')) \\ - 2\, \mathbb{E}_{G \sim \mathcal{P}, H \sim \mathcal{Q}}\, k(\phi(G), \phi(H)).\end{aligned} \tag{2.21}$$

Dato un campione osservato $\{G_i\}_{i=1}^n$ e uno generato $\{H_j\}_{j=1}^m$, lo stimatore *non distorto* [9] rimuove i termini diagonali di auto-confronto:

$$\begin{aligned}\widehat{\mathrm{MMD}}_k^2 = \frac{1}{n(n-1)} \sum_{i \neq i'} k(\phi(G_i), \phi(G_{i'})) + \frac{1}{m(m-1)} \sum_{j \neq j'} k(\phi(H_j), \phi(H_{j'})) \\ - \frac{2}{nm} \sum_{i=1}^{n} \sum_{j=1}^{m} k(\phi(G_i), \phi(H_j)).\end{aligned} \tag{2.22}$$

Essendo non distorto, lo stimatore (2.22) puo' assumere valori negativi; e' inoltre uno stimatore di MMD^2, non di MMD.

I kernel considerati nella pipeline sono il kernel lineare $k(a, b) = \langle a, b \rangle$, privo di iperparametri, e il kernel gaussiano

$$k_\sigma(a, b) = \exp\bigl(-\sigma \, \|a - b\|_2^2\bigr). \tag{2.23}$$

Si noti che qui σ parametrizza una *precisione* e non una larghezza di banda. GGME sconsiglia esplicitamente il kernel basato su Earth Mover's Distance (troppo costoso, fino a due ordini di grandezza piu' lento) e il kernel gaussiano basato sulla distanza di variazione totale, che non e' definito positivo perche' lo spazio metrico indotto dalla TV non e' piatto (Teorema 1 di GGME).

2.5.3 Selezione di kernel e iperparametri

Il contributo metodologico principale di GGME e' la dimostrazione che il ranking dei modelli e' *arbitrario* rispetto alla scelta di kernel, σ e numero di bin del descrittore: qualunque modello puo' risultare primo con una scelta opportuna. La soluzione proposta e' selezionare tali elementi mediante un esperimento di perturbazione controllata che *non coinvolge i modelli*:

1. si genera una sequenza di versioni perturbate del test set con intensita' crescente $p_1 < p_2 < \cdots < p_L$;

2. per ogni combinazione (kernel, σ) si calcola $\widehat{\mathrm{MMD}}^2$ tra test set e ciascuna versione perturbata;

3. si sceglie la combinazione che massimizza la correlazione di Pearson tra l'intensita' della perturbazione e la MMD risultante.

La procedura e' non distorta rispetto ai modelli in confronto, perche' dipende solo dalla distribuzione di partenza. GGME raccomanda inoltre di riportare la MMD tra training e test set come *scala di riferimento*, dato che la MMD non ha una scala intrinseca.

Capitolo 3

La pipeline OGAD

Questo capitolo descrive la pipeline implementata, collegando ciascuna scelta implementativa al corrispondente formalismo del Capitolo 2. La struttura complessiva e'

$$\underbrace{\text{SIGL}}_{\text{stadio 1}} \longrightarrow \underbrace{\text{D3PM } \textit{oppure} \text{ DiGress su graphon}}_{\text{stadio 2}} \longrightarrow \underbrace{\text{campionamento di grafi}}_{\text{stadio 3}} \longrightarrow \underbrace{\text{GGME}}_{\text{stadio 4}}.$$

Formalmente, indicando con $\mathcal{D}_{\text{train}} = \{G_1, \dots, G_N\}$ il training set di grafi, il primo stadio produce una collezione di graphon stimati

$$\widehat{\mathcal{W}} = S_\psi(\mathcal{D}_{\text{train}}) = \{\widehat{W}_1, \dots, \widehat{W}_L\}, \qquad \widehat{W}_\ell \in [0,1]^{R \times R}, \tag{3.1}$$

dove S_ψ rappresenta lo stimatore SIGL. Il secondo stadio apprende una distribuzione $p_\theta(W)$ su tale collezione e ne estrae campioni $\widetilde{W}_1, \dots, \widetilde{W}_S \sim p_\theta$. Il terzo stadio induce una distribuzione su grafi

$$\widetilde{G} \sim \mathbb{P}\big(\cdot \mid \Pi(\widetilde{W}), M\big), \qquad M \sim \hat{\mu}_M, \tag{3.2}$$

con $\hat{\mu}_M$ la distribuzione empirica delle dimensioni dei grafi di test. Il quarto stadio confronta $\mathbb{P}_{\mathcal{G}}^{\text{test}}$ con $\mathbb{P}_{\mathcal{G},\theta,\psi}^{\text{OGAD}}$.

3.1 Dataset

Tutti gli esperimenti usano il dataset *Community Graphs*, distribuito con il repository GGME e memorizzato in formato pickle:

File	N grafi	Nodi	Formato
`data/train/graphs/CommunityGraphs.pickle`	400	45	`csr_matrix`
`data/test/graphs/CommunityGraphs.pickle`	100	45	`csr_matrix`
`data/predictions/graphs/ggme_paper/`	400	44–45	`ndarray`

La densita' media di archi e' ≈ 0.234 sul training set e ≈ 0.232 sul test set. Il fatto che *tutti* i grafi abbiano esattamente 45 nodi ha una conseguenza importante sul terzo stadio: la distribuzione empirica $\hat{\mu}_M$ e' degenere in 45, quindi tutti i grafi generati hanno 45 nodi e le differenze nelle metriche non possono essere imputate a una diversa distribuzione delle dimensioni.

Il terzo file, indicato nel confronto come «Paper», contiene le predizioni di esempio distribuite con il repository GGME: sono i grafi prodotti da un modello generativo di riferimento, e fungono da soglia di qualita' esterna.

3.2 Stadio 1: stima dei graphon con SIGL

3.2.1 Il problema della cardinalita' del training set

SIGL, come formulato in [2], produce *un solo* graphon a partire da un insieme di grafi. Questo e' incompatibile con lo stadio 2, che richiede un dataset di graphon sufficientemente numeroso per addestrare un modello di diffusione. La pipeline OGAD risolve il problema con una procedura in due passi: *clustering* dei grafi seguito da *bootstrap non parametrico* all'interno di ciascun cluster.

Clustering. Ogni grafo G_i e' mappato in un vettore di feature ottenuto concatenando i *medesimi tre descrittori* usati nella valutazione GGME, in forma normalizzata:

$$\mathbf{f}(G) = \big[\, \phi_{\deg}(G) \parallel \phi_{\text{clust}}(G) \parallel \phi_{\text{lap}}(G) \,\big]. \tag{3.3}$$

I vettori vengono zero-padded a lunghezza comune, standardizzati, e partizionati con k-means in C cluster ($C = 50$ nella configurazione principale, $C = 200$ in un test preliminare). Si ottiene una partizione $\{\mathcal{I}_1, \dots, \mathcal{I}_C\}$ degli indici dei grafi.

Bootstrap. Per ogni cluster c e per ogni replica $r = 1, \dots, B$ si estrae un campione bootstrap di indici

$$\mathcal{I}_c^{(r)} = \big(i_1^{(r)}, \dots, i_{|\mathcal{I}_c|}^{(r)}\big), \qquad i_s^{(r)} \overset{\text{i.i.d.}}{\sim} \text{Uniform}(\mathcal{I}_c), \tag{3.4}$$

cioe' $|\mathcal{I}_c|$ estrazioni *con reinserimento* dai membri del cluster. Il numero totale di graphon prodotti e'

$$L = \begin{cases} C, & B = 1, \\ C \cdot B + C \mathbb{1}[\text{include_original}], & B > 1. \end{cases} \tag{3.5}$$

Con $C = 50$ e $B \in \{12, 13, 14, 15, 16\}$ si ottengono $L \in \{600, 650, 700, 750, 800\}$ graphon. Ogni job di bootstrap innesca una stima SIGL *completamente indipendente*: sia la GCN sia la SIREN sono reinizializzate da zero, cosicche' i L graphon sono L stime distinte e non L viste dello stesso oggetto.

Osservazione 3.1 (Interpretazione statistica)**.** La procedura puo' essere letta come una stima della *distribuzione campionaria* dello stimatore SIGL, condizionata alla struttura locale del dataset. Il clustering identifica sottopopolazioni omogenee rispetto ai descrittori strutturali; il bootstrap all'interno di ciascuna genera la variabilita' che il modello diffusivo deve poi apprendere. Senza bootstrap, il modello vedrebbe C graphon quasi deterministici; senza clustering, la variabilita' bootstrap sarebbe dominata dall'eterogeneita' globale del dataset.

Split di validazione. Prima del clustering, il training set viene suddiviso in modo deterministico (seed 0, frazione 0.2) in 320 grafi usati per la stima dei graphon e 80 grafi tenuti da parte come *validation set GGME*, impiegato nello stadio 4 per la selezione del candidato migliore (Sezione 3.7.3). Questo evita che la selezione del modello usi il test set.

3.2.2 Iperparametri e architettura effettivi

Componente	Iperparametro	Valore
GCN g_{ϕ_1}	strati `GCNConv`	3, con 8 unita' ciascuno
	attivazione	ReLU, poi Linear($8 \to 1$) + sigmoide
	normalizzazione	min–max per grafo, in $[0, 1]$
	feature di nodo	$x_i \sim \mathcal{N}(0, 1)$, scalari
SIREN h_{ϕ_2}, f_θ	strati nascosti	2, con 20 unita'
	ω_0 primo strato	10
	ω_0 strati successivi	1
	attivazione finale	sigmoide
Ottimizzazione	epoche (passi 1 e 3)	100
	learning rate	0.06 (Adam)
	batch (passo 1)	1 grafo
	batch (passo 3)	1024 coordinate
Discretizzazione	finestra di pooling h	$\lfloor \ln n \rfloor = 3$
	risoluzione istogramma k	$\lfloor 45/3 \rfloor = 15$
	risoluzione di uscita R	128

Per $n = 45$ nodi il passo 2 produce quindi istogrammi 15×15, ossia 225 coppie $(x, y, \hat{H})$ per grafo. Poiche' tutti i grafi hanno la stessa dimensione, i pesi w_t di (2.7) sono uniformi e la loss pesata degenera in un MSE ordinario.

3.2.3 Formato di uscita

Il graphon stimato viene campionato dalla INR su una griglia regolare di celle centrate,

$$\widehat{W}_{ij} = f_{\theta^\star}\left(\frac{i+\frac{1}{2}}{R}, \frac{j+\frac{1}{2}}{R}\right), \qquad i, j = 0, \dots, R-1, \tag{3.6}$$

quindi sanificato secondo (2.4) e salvato come immagine PNG in scala di grigio a 8 bit, con nome `sample_{ℓ}.png`. Un file `bootstrap_metadata.json` registra, per ciascun graphon, il cluster di origine, l'identificativo di replica e gli indici dei grafi effettivamente usati.

Osservazione 3.2 (Quantizzazione implicita a 8 bit)**.** Il passaggio attraverso il formato PNG introduce una quantizzazione non documentata nella formulazione teorica: i valori vengono arrotondati a $\mathrm{round}(255\,\widehat{W}_{ij})/255$. Poiche' lo stadio 2 applica una seconda quantizzazione a $K \approx 10$ livelli, la risoluzione a 8 bit non e' il fattore limitante; resta tuttavia una scelta implementativa da segnalare, perche' rende il formato di scambio tra stadio 1 e stadio 2 lossy e non esattamente invertibile.

3.3 Stadio 2, variante A: OGAD–D3PM

3.3.1 Rappresentazione del dato

Il graphon e' trattato come un'*immagine categoriale* a canale singolo. Dato $\widehat{W} \in [0,1]^{R\times R}$, la quantizzazione e'

$$b(w) = \mathrm{clip}\Big(\big\lfloor w\,(K-1)\big\rceil,\ 0,\ K-1\Big) \ \in\ \{0, \dots, K-1\}, \tag{3.7}$$

dove $\lfloor\cdot\rceil$ denota l'arrotondamento al pari intero piu' vicino. La dequantizzazione e' esattamente l'inversa:

$$b^{-1}(k) = \frac{k}{K-1}. \tag{3.8}$$

Si osservi che questa non e' una discretizzazione a bin di uguale ampiezza, ma un arrotondamento al piu' vicino fra K *livelli equispaziati* su $[0,1]$. I

livelli sono $\{0, \frac{1}{K-1}, \ldots, 1\}$ e i confini impliciti sono i punti medi $\frac{2k+1}{2(K-1)}$; i due bin estremi hanno quindi ampiezza dimezzata rispetto a quelli interni.

Il modello opera sull'intera matrice $R \times R = 128 \times 128$: non vi e' estrazione del triangolo superiore ne' mascheramento della diagonale. Le 16 384 celle sono trattate come variabili categoriali condizionalmente indipendenti dato x_t, e la simmetria e' un vincolo *soft*, ereditato dai dati e reimposto a posteriori tramite (2.4). Questo comporta uno spreco di capacita' pari a circa un fattore 2, ed e' una delle direzioni di miglioramento piu' immediate della pipeline.

3.3.2 Processo diffusivo

- **Matrice di transizione**: uniforme, secondo (2.12), con K classi. La prior a $t = T$ e' dunque uniforme su $\{0, \ldots, K-1\}$, coerentemente con la distribuzione stazionaria del processo.

- **Schedule**: coseno secondo (2.13) con esponente $p = 1$, troncato superiormente:
$$\beta_t = \min\left(1 - \frac{\bar{\alpha}(t/T)}{\bar{\alpha}((t-1)/T)},\ 0.999\right).$$

- **Passi**: $T = 1000$, fissato nel codice e non esposto come iperparametro di configurazione.

- **Cumulate**: le $\overline{Q}_t$ sono precalcolate esplicitamente come prodotti di matrici $K \times K$ e memorizzate come buffer; il costo di memoria e' $O(TK^2)$, trascurabile per $K \leq 14$.

- **Campionamento**: sia in avanti che all'indietro il campionamento categoriale e' realizzato con il *Gumbel-max trick*, $x = \arg\max_i(\log \pi_i + g_i)$ con $g_i \sim \text{Gumbel}(0, 1)$. All'ultimo passo ($t = 1$) il rumore di Gumbel viene azzerato, rendendo l'ultima transizione deterministica.

Durante l'addestramento i timestep sono estratti uniformemente in $\{1, \ldots, T-1\}$, quindi $t = T$ non viene mai visitato.

3.3.3 Rete di denoising

Il denoiser e' un *Diffusion Transformer* in stile DiT [16] con blocchi di tipo LLaMA [24], denominato `DiT_Llama` nel repository [18]:

Componente	Configurazione
Stem convoluzionale	$2 \times$ `Conv2d`$(k = 5)$ + SiLU + GroupNorm, $1 \to d/2$ canali
Patchify	patch $2 \times 2 \Rightarrow 64^2 = 4096$ token
Larghezza d	512
Blocchi transformer	5, con 16 teste
Attenzione posizionale	RoPE [23]
Feed-forward	SwiGLU
Condizionamento su t	adaLN-Zero da embedding sinusoidale (256 frequenze) + MLP
Testa finale	`Linear` inizializzata a zero, $d \to 4 \cdot 2K$

L'input intero viene riscalato in $[-1, 1]$ tramite $\tilde{x} = 2x/(K-1) - 1$, e l'uscita e' sommata a una versione *gated* della codifica one-hot dell'input:

$$\text{logits}(x_t) = \text{nn}_\theta(x_t) + \text{onehot}(x_t) \odot \big|1 + \gamma_\theta(x_t)\big|, \tag{3.9}$$

cioe' la rete apprende un residuo attorno al denoiser identita'. Le shape sono: input $(B, 1, 128, 128)$ interi, output $(B, 1, 128, 128, K)$ logit.

3.3.4 Addestramento

La loss e' quella ibrida (2.16), con il termine variazionale calcolato come media (non somma) delle KL su tutte le 16 384 posizioni. La griglia di iperparametri esplorata e':

Iperparametro	Valori
`lr`	$\{2, 3, 4, 5\} \times 10^{-5}$
`num_classes` K	$\{10, 12, 14\}$
`hybrid_loss_coeff` λ	$\{0.006, 0.01, 0.015\}$
`batch_size`	32
`dim` d	512

per un totale di $4 \times 3 \times 3 = 36$ candidati per ogni configurazione SIGL. L'ottimizzatore e' AdamW con clipping della norma del gradiente a 5.0 e mixed precision; non e' usata alcuna media esponenziale dei pesi. Il protocollo di early stopping monitora la loss di validazione (frazione 0.2, split deterministico con seed 0) con `patience` $= 25$, `min_delta` $= 5 \times 10^{-5}$ e al piu' 300 epoche; i pesi dell'epoca migliore vengono ripristinati.

Osservazione 3.3. La loss di validazione e' essa stessa stocastica, perche' a ogni valutazione vengono riestratti sia t sia il rumore di Gumbel. Il segnale di early stopping e' quindi rumoroso, il che suggerisce di mediare su piu' passate o di fissare i timestep di validazione come miglioria futura.

3.3.5 Generazione

Il campionamento parte da rumore uniforme $x_T \sim \text{Uniform}\{0, \dots, K-1\}^{16\times1\times128\times128}$ e itera la transizione inversa (2.14) per $t = T-1, \dots, 1$, cioe' 999 passi. Il tensore finale viene dequantizzato secondo (3.8) e sanificato secondo (2.4), producendo

`data/predictions/graphons/tensors.pt` di forma $[16, 1, 128, 128]$.

3.4 Stadio 2, variante B: OGAD–DiGress

3.4.1 Riduzione del graphon a un grafo etichettato

L'idea centrale di questa variante e' che un graphon quantizzato *e'* un grafo con archi categoriali. Formalmente si definisce l'immersione

$$\Phi_K : [0,1]^{R\times R} \longrightarrow \mathcal{G}_{R,1,K}, \qquad \Phi_K(\widehat{W}) = \big(X, E\big), \tag{3.10}$$

dove:

- il grafo sottostante e' il *grafo completo* K_R su $R = 128$ nodi, privo di self-loop, con $R(R-1) = 16\,256$ archi orientati;
- $X = \mathbf{1}_{R\times1}$, cioe' vi e' un *unico* tipo di nodo ($a = 1$) e i nodi non portano informazione;
- $E_{ij} \in \{0,1\}^K$ e' la codifica one-hot della classe

$$\ell_{ij} = \big\lfloor \Pi(\widehat{W})_{ij}\,(K-1) \big\rceil \;\in\; \{0, \dots, K-1\}, \tag{3.11}$$

  con $b = K$ tipi di arco;
- y e' vuoto (0 feature globali).

La quantizzazione (3.11) e' formalmente identica alla (3.7) di D3PM: entrambe le varianti condividono lo stesso schema di discretizzazione, il che rende il confronto tra i due generatori significativo. Con $K = 10$ i livelli rappresentabili sono $\{0, \frac{1}{9}, \frac{2}{9}, \dots, 1\}$, ossia un passo di quantizzazione ≈ 0.111.

Osservazione 3.4 (Che cosa cambia rispetto a DiGress standard)**.** In DiGress applicato ai grafi molecolari, la classe di arco 0 rappresenta *l'assenza di arco* e le altre classi i tipi di legame; il grafo e' sparso. Qui, al contrario, la classe 0 rappresenta la probabilita' $W_{ij} = 0$ ed e' una classe come le altre: il grafo di supporto e' sempre completo. Di conseguenza la distribuzione dei nodi e' degenere ($n = 128$ con probabilita' 1) e il conteggio delle classi d'arco viene calcolato includendo esplicitamente il bin zero, invece di riservarlo ai non-archi come nell'implementazione originale.

3.4.2 Processo diffusivo

Poiche' $a = 1$, la matrice Q_X^t e' l'identita' 1×1 e il ramo dei nodi e' inerte. Tutta la dinamica si svolge sul tensore degli archi, con transizione **uniforme**:

$$Q_E^t = (1 - \beta_t)\, I_K + \frac{\beta_t}{K}\, \mathbf{1}\mathbf{1}^\top, \qquad \overline{Q}_E^t = \bar{\alpha}_t\, I_K + (1 - \bar{\alpha}_t)\, \frac{\mathbf{1}\mathbf{1}^\top}{K}, \tag{3.12}$$

con distribuzione limite uniforme sui K bin. Lo schedule e' coseno secondo (2.13) con esponente $p = 2$ e $T = 1000$.

Osservazione 3.5 (Uniforme anziche' marginale)**.** Questa e' una deviazione consapevole dal default di DiGress, che usa la transizione marginale. Il codice della pipeline calcola comunque la distribuzione marginale empirica dei bin (includendo il bin zero, come detto sopra), ma con la configurazione `transition: uniform` tale quantita' non viene consumata. La motivazione della scelta e' la comparabilita' con OGAD–D3PM, che usa anch'esso una transizione uniforme; attivare la variante marginale richiede la sola modifica del campo di configurazione ed e' la prima ablation naturale da eseguire, dato che il Teorema 4.1 di DiGress ne prevede un vantaggio.

Il rumore e' applicato al solo triangolo superiore di E e poi simmetrizzato, quindi in questa variante — a differenza di OGAD–D3PM — la simmetria del graphon e' garantita *per costruzione* lungo tutta la traiettoria diffusiva.

3.4.3 Rete di denoising e loss

Viene usato il `GraphTransformer` di DiGress [25, 7] senza alcuna modifica: 5 strati che aggiornano congiuntamente le feature di nodo X, di arco E e globali y, con condizionamento reciproco tramite layer FiLM [17] e aggregazione PNA per le feature globali. Le dimensioni effettive sono hidden_mlp $= \{X{:}128, E{:}64, y{:}128\}$ e hidden $= \{d_X{:}256, d_E{:}64, d_y{:}64,\ n_{\text{head}}{:}8\}$.

Le **feature ausiliarie strutturali e spettrali sono disattivate**. La scelta e' teoricamente motivata: il grafo di supporto e' una cricca su 128 nodi identica per tutti i campioni, dunque conteggi di cicli, numero di componenti connesse e spettro del laplaciano del supporto sono costanti e non veicolano alcuna informazione. Le dimensioni di input della rete sono quindi $X{:}1$, $E{:}K$, $y{:}1$ (il singolo canale globale codifica il timestep normalizzato).

La loss e' la (2.18) con $\lambda = 5$. Poiche' $a = 1$, la softmax sui tipi di nodo e' degenere e $\mathrm{CE}_X \equiv 0$; l'obiettivo effettivamente ottimizzato e' dunque

$$l(\hat{p}^G, G) = 5 \sum_{1 \le i,j \le R} \mathrm{CE}\big(e_{ij}, \hat{p}_{ij}^E\big), \tag{3.13}$$

ossia una pura cross-entropy sui K bin di probabilita'. Il criterio di selezione del checkpoint e' invece la NLL variazionale di validazione (`val/epoch_NLL`), corrispondente al bound (2.15).

3.4.4 Addestramento e generazione

Iperparametro	Valore
`batch_size`	2
`accumulate_grad_batches`	16 (batch efficace 32)
`n_epochs`	200
`lr`	2×10^{-4}
ottimizzatore	AdamW, `amsgrad`, wd $= 10^{-12}$
`n_layers`	5
`num_bins` K	10
`resolution` R	128
campioni finali	16 (100 nella configurazione estesa)
EMA	disattivata

Lo split dei graphon e' 80/10/10 deterministico. Il campionamento parte da rumore uniforme sui bin, gia' simmetrizzato con diagonale nulla, e collassa il tensore finale con $\arg\max$ sui K canali. La post-elaborazione consiste in:

1. **dequantizzazione** $\ell \mapsto \ell/(K-1)$ — si usa *solo* il tensore degli archi E, i tipi di nodo predetti sono scartati;
2. **sanificazione** secondo (2.4);
3. **correzione di densita'** opzionale, attiva nella configurazione corrente: detta
$$\rho(W) = \frac{1}{R(R-1)} \sum_{i \neq j} W_{ij} \tag{3.14}$$
la densita' fuori diagonale e $\rho^\star$ la densita' media dei graphon di training prodotti da SIGL, ogni graphon generato viene riscalato come
$$W \;\longleftarrow\; \Pi\!\left(\frac{\rho^\star}{\rho(W)}\, W \right). \tag{3.15}$$

Osservazione 3.6. La correzione (3.15) e' un intervento *ad hoc* e va riportato come tale: essa forza un momento del primo ordine del graphon generato a coincidere con quello dei dati, ma non e' parte del modello probabilistico. Inoltre, per via del clipping in $[0, 1]$ contenuto in Π, quando il fattore

di scala supera l'unita' la densita' ottenuta sottostima sistematicamente il bersaglio. La pipeline registra le densita' prima e dopo la correzione in un file `stats.json` per rendere l'effetto verificabile.

L'uscita e' `data/predictions/graphons/digress/tensors.pt`, di forma $[N, 1, 128, 128]$, formato identico a quello prodotto dalla variante D3PM: i due generatori sono quindi intercambiabili rispetto agli stadi 3 e 4.

3.5 Confronto tra le due varianti

	OGAD–D3PM	**OGAD–DiGress**
Rappresentazione	immagine categoriale 128×128	grafo completo K_{128} con archi a K classi
Quantizzazione	$\lfloor w(K-1) \rceil$	$\lfloor w(K-1) \rceil$ (identica)
K	$\{10, 12, 14\}$	10
Entita' modellate	tutte le 16 384 celle	triangolo superiore, 8 128 archi
Simmetria	soft, reimposta a posteriori	garantita per costruzione
Transizione Q_t	uniforme	uniforme (marginale disponibile)
Schedule	coseno, $p = 1$, $T = 1000$	coseno, $p = 2$, $T = 1000$
Denoiser	DiT/LLaMA, 5 blocchi, $d = 512$, RoPE	GraphTransformer, 5 strati, $d_X = 256$, $d_E = 64$
Induttivo	localita' 2D via patch e convoluzioni	equivarianza a permutazione
Loss	$L_{\text{vb}} + \lambda\,\text{CE}$	$5 \cdot \text{CE}_E$
Feature ausiliarie	assenti	disattivate esplicitamente
Selezione modello	griglia di 36 candidati + MMD	checkpoint a NLL minima
Post-processing	sanificazione	sanificazione + density matching

La differenza concettuale piu' rilevante riguarda il *bias induttivo*. Il denoiser di D3PM tratta il graphon come un segnale bidimensionale e sfrutta

la localita' spaziale attraverso convoluzioni e patch: questo e' coerente con il fatto che, dopo l'ordinamento SIGL, i graphon dei grafi Community presentano una struttura a blocchi *localmente regolare*. Il denoiser di DiGress e' invece equivariante a permutazione, quindi indifferente all'ordinamento delle righe e delle colonne; poiche' l'ordinamento e' esattamente l'informazione che SIGL ha faticato a produrre, l'equivarianza rappresenta in questo contesto piu' un vincolo che un vantaggio. Ci si attende quindi che la variante D3PM sia avvantaggiata sui graphon ordinati, e questa e' un'ipotesi verificabile sperimentalmente.

Un secondo aspetto riguarda la scala dei dati. Con $C = 50$ e $B = 13$ SIGL produce 650 graphon, di cui 80% usati per il training. La variante D3PM, con batch 32, esegue circa 16 passi di ottimizzazione per epoca; la variante DiGress, con batch 2 e accumulo 16, ne esegue circa 16 altrettanti ma su un modello con costo per campione molto maggiore, dato che il GraphTransformer deve gestire un tensore di archi con 16 256 entita' per grafo.

3.6 Stadio 3: campionamento dei grafi

Sia $\mathcal{D}_{\text{test}} = \{G_1^{\text{test}}, \dots, G_n^{\text{test}}\}$ il test set e $m_i = |V(G_i^{\text{test}})|$ le dimensioni dei grafi osservati. La distribuzione empirica delle dimensioni e'

$$\hat{\mu}_M = \frac{1}{n} \sum_{i=1}^{n} \delta_{m_i}. \tag{3.16}$$

Per generare il campione r-esimo, la pipeline seleziona ciclicamente un graphon generato,

$$\widetilde{W}^{(r)} = \widetilde{W}_{1+((r-1) \bmod S)}, \tag{3.17}$$

estrae $M_r \sim \hat{\mu}_M$ con rigetto se $M_r > R$, e campiona il grafo secondo (2.1)–(2.3):

$$\eta_1, \dots, \eta_{M_r} \overset{\text{i.i.d.}}{\sim} \text{Uniform}([0,1]), \quad z_i = \lfloor R\eta_i \rfloor, \quad A_{ij}^{(r)} \sim \text{Bernoulli}\Big(\Pi(\widetilde{W}^{(r)})_{z_i z_j}\Big) \ \ (i < j), \tag{3.18}$$

simmetrizzando poi $A^{(r)}$. Il dataset generato usato in valutazione e' $\widetilde{\mathcal{D}} = \{\widetilde{G}_1, \dots, \widetilde{G}_B\}$ con $B = 100$ e seed fissato a 0.

Due dettagli meritano attenzione. Primo, la selezione *ciclica* anziche' casuale dei graphon garantisce un utilizzo bilanciato: con $S = 16$ graphon e $B = 100$ grafi, i primi 4 graphon vengono usati 7 volte e i restanti 6. Secondo, si usa un *unico* vettore di variabili latenti η per entrambi gli assi, il che e' la formulazione corretta di (2.1): campionare due vettori indipendenti

produrrebbe una matrice non simmetrica e non corrisponderebbe al modello W-random graph.

Poiche' nel dataset Community tutti i grafi di test hanno 45 nodi e $R = 128$, si ha $M_r = 45$ deterministicamente e il rigetto non si attiva mai.

3.7 Stadio 4: valutazione GGME

3.7.1 Descrittori

Sono implementati tre descrittori, tutti calcolati in forma normalizzata:

Descrittore	Binning	Normalizzazione
ϕ_{deg}: istogramma dei gradi	lunghezza variabile ($\max \deg +1$)	somma unitaria (pmf)
ϕ_{clust}: coeff. di clustering	200 bin su $[0, 1]$	integrale unitario
ϕ_{lap}: spettro laplaciano norm.	200 bin su $[0, 2]$	integrale unitario

L'istogramma dei gradi ha lunghezza variabile e viene zero-padded alla lunghezza massima prima del calcolo del kernel.

Osservazione 3.7 (Scala delle metriche)**.** I descrittori a binning fisso sono normalizzati come *densita'* e non come distribuzioni di probabilita': l'integrale, non la somma, e' unitario. Il fattore di scala e' quindi 200 per il coefficiente di clustering (200 bin su un intervallo di ampiezza 1) e 100 per lo spettro (200 bin su ampiezza 2), mentre l'istogramma dei gradi e' una pmf ordinaria. Poiche' un kernel lineare e' omogeneo di grado 2, i valori di MMD del clustering risultano scalati di un fattore $\approx 200^2 = 4 \times 10^4$ rispetto a quelli del grado. **Questo spiega interamente** l'ordine di grandezza radicalmente diverso delle tre colonne nei risultati (Capitolo 4) e implica che i valori non sono confrontabili *tra* descrittori, ma solo *tra* modelli a parita' di descrittore.

3.7.2 Protocollo di selezione

Il protocollo GGME e' implementato come segue:

1. **Perturbazione**: si applica la perturbazione `AddEdges`, che aggiunge ogni coppia non adiacente in modo indipendente con probabilita' p, a 17 livelli di intensita' $p \in \{0.15, 0.20, \ldots, 0.95\}$ (seed 42). Ogni livello produce una copia perturbata dei 100 grafi di test.

2. **Griglia**: per ciascun descrittore si esplorano $\{\text{gaussiano}, \text{lineare}\} \times \{\sigma = 0.1, 1, 10\}$.

3. **Selezione**: si sceglie la combinazione che massimizza la correlazione di Pearson fra il livello di perturbazione p e la $\widehat{\mathrm{MMD}}^2$ risultante.

4. **Valutazione**: con la configurazione selezionata si calcola $\widehat{\mathrm{MMD}}^2$ fra test set e ciascuno dei tre insiemi in confronto.

Le configurazioni selezionate empiricamente sono riportate in Tabella 3.2.

Tabella 3.2: Configurazioni di kernel selezionate dal protocollo di perturbazione.

Descrittore	Kernel	σ	Correlazione
ϕ_{deg}	gaussiano	10	0.8946
ϕ_{clust}	lineare	—	0.7638
ϕ_{lap}	lineare	—	0.7112

3.7.3 Selezione del candidato D3PM tramite MMD

Nella variante D3PM, oltre alla selezione per loss di validazione, e' previsto un secondo stadio di selezione basato sulle metriche. Per ciascuno dei 36 candidati si generano 100 grafi dai rispettivi graphon e si calcolano le tre MMD rispetto agli 80 grafi tenuti da parte nello split di Sezione 3.2. Il punteggio usato e' il rapporto medio rispetto a un bersaglio fisso:

$$\mathrm{score}(c) = \frac{1}{3} \sum_{d \in \{\mathrm{deg,clust,lap}\}} \frac{\widehat{\mathrm{MMD}}^2_d(c)}{\tau_d}, \qquad \tau = (0.0125,\ 99.7936,\ 1.8530), \tag{3.19}$$

dove i valori τ_d sono le MMD ottenute da DiGress applicato direttamente ai grafi. Il candidato con punteggio minimo viene promosso e i suoi graphon sovrascrivono `tensors.pt`.

Osservazione 3.8. La normalizzazione per τ_d e' necessaria proprio a causa dell'Osservazione 3.7: senza di essa, la media aritmetica delle tre MMD sarebbe interamente dominata dal termine di clustering. La scelta di usare come normalizzatore i valori di DiGress rende il punteggio interpretabile (score < 1 significa «mediamente meglio di DiGress»), ma introduce anche una dipendenza da una esecuzione specifica del baseline, che andrebbe idealmente sostituita da una normalizzazione intrinseca, ad esempio la MMD fra training e test set raccomandata da GGME.

3.7.4 Termini di confronto

Lo stadio GGME confronta tre insiemi di grafi con il test set:

- **DiGress**: grafi generati da DiGress *vanilla* applicato direttamente ai grafi Community. Questa variante usa 2 classi di arco (presenza/assenza), transizione marginale, 8 strati, feature ausiliarie strutturali e spettrali attive, e produce 160 campioni. Nel flusso orchestrato tramite script questo stadio viene saltato (`--skip-digress`) e la colonna corrispondente risulta vuota; i valori riportati provengono da un'esecuzione dell'intero stack `docker compose`.
- **OGAD**: i 100 grafi campionati dai graphon generati secondo Sezione 3.6.
- **Paper**: i 400 grafi di riferimento distribuiti con GGME.

Osservazione 3.9. I tre insiemi non hanno la stessa cardinalita' (160, 100, 400). Lo stimatore non distorto (2.22) gestisce correttamente $n \neq m$, ma la varianza della stima differisce fra le tre colonne. La pipeline impone un controllo di consistenza fra DiGress e OGAD, ma non fra questi e il riferimento Paper: uniformare le cardinalita' e' una delle correzioni previste al protocollo.

3.8 Orchestrazione e riproducibilita'

Ogni stadio e' incapsulato in un servizio Docker distinto, con immagini separate: `ogad-prep-data` (SIGL), `ogad-train-model-b` (D3PM) oppure `ogad-train-model-b-digress` (DiGress su graphon), `ogad-train-model-a` (DiGress baseline) e `ogad-eval-metrics` (GGME). Lo script `scripts/run_ogad_experiment.sh` esegue in sequenza:

La configurazione sperimentale e' descritta da un unico file JSON i cui valori sono liste; il blocco SIGL viene espanso in prodotto cartesiano generando run indipendenti, mentre il blocco D3PM resta una griglia interna a ciascuna run. Un driver parallelo distribuisce le run su piu' GPU con assegnazione round-robin. La riproducibilita' e' garantita fissando i seed di `random`, `numpy` e `torch` per ciascun candidato, e usando split deterministici basati su seed espliciti in tutti i punti di partizionamento dei dati.

Algorithm 1 Esecuzione di un esperimento OGAD

```
RUN_ID ← timestamp UTC; crea la directory di run
scrive il manifesto e copia la configurazione JSON in runs/current/
seleziona lo stack compose in base a OGAD_GRAPHON_GENERATOR ∈ {d3pm, digress}
stadio 1                                                  ▷ servizio prep_data
  python Main.py --config $CONFIG
stadio 2                                                  ▷ servizio train_model_b
if generatore = d3pm then
    python ogad_model_finetune.py --config $CONFIG
    python ogad_model_inference.py
else
    python src/main.py +experiment=graphons {override Hydra}
end if
stadi 3–4                                                 ▷ servizio eval_metrics
  python main.py --config $CONFIG --skip-digress
scrive log e metrics.jsonl nella directory di run
```

Capitolo 4

Risultati preliminari

4.1 Risultati riportati

La Tabella 4.1 riporta i valori di $\widehat{\mathrm{MMD}}^2$ rispetto al test set per la configurazione con $C = 200$ cluster, insieme a una seconda esecuzione con $C = 50$ e bootstrap.

Tabella 4.1: $\widehat{\mathrm{MMD}}^2$ rispetto al test set. Valori piu' bassi indicano maggiore somiglianza. Si veda l'Osservazione 3.7 per la scala.

Descrittore	DiGress	OGAD	Paper
Configurazione $C = 200$*, senza bootstrap*			
ϕ_{deg}	0.0125	0.6301	0.0034
ϕ_{clust}	99.7936	2458.3862	16.0080
ϕ_{lap}	1.8530	44.1202	0.5686
Configurazione $C = 50$*, con bootstrap*			
ϕ_{deg}	—	0.1562	0.0034
ϕ_{clust}	—	215.876	16.008
ϕ_{lap}	—	2.8238	0.5686

4.2 Lettura dei risultati

Il primo dato rilevante e' che l'introduzione del clustering con bootstrap ($C = 50$, $B \geq 12$) migliora OGAD di un fattore compreso fra 4 e 15 su tutti e tre i descrittori rispetto alla configurazione senza bootstrap. Questo

e' coerente con l'interpretazione della Sezione 3.2: aumentare la cardinalita' e la variabilita' del dataset di graphon migliora la stima di $p_\theta(W)$.

Il secondo dato e' che OGAD resta comunque distante dal riferimento Paper di circa un ordine di grandezza su ciascun descrittore. Il divario e' particolarmente marcato sul coefficiente di clustering, che misura la tendenza triadica. Questo e' prevedibile dalla struttura del modello: il campionamento (3.18) genera archi *condizionatamente indipendenti* dati i latenti, quindi il numero atteso di triangoli e' completamente determinato da W e la varianza triadica dei grafi generati e' quella minima compatibile con il modello W-random graph. Se il dataset reale contiene correlazioni di ordine superiore non catturabili da un graphon, nessun miglioramento del generatore diffusivo potra' colmarle: si tratterebbe di un limite del *modello* e non dell'*algoritmo*.

Distinguere fra le due spiegazioni — limite del graphon come modello, oppure errore di stima e generazione — e' l'esperimento diagnostico piu' importante da eseguire. Il protocollo naturale e' calcolare la MMD fra il test set e i grafi campionati direttamente dai graphon *stimati* da SIGL, senza passare per il generatore diffusivo. Tale valore costituisce un *floor* raggiungibile: se e' vicino a quello di OGAD, il collo di bottiglia e' lo stadio 1; se e' molto piu' basso, lo e' lo stadio 2.

4.3 Ipotesi sulle sorgenti di errore

L'analisi implementativa del Capitolo 3 suggerisce un elenco ordinato di cause candidate:

1. **Risoluzione della quantizzazione.** Con $K = 10$ il passo di quantizzazione e' $1/9 \approx 0.111$. Per un dataset la cui densita' media e' 0.234, questo significa che l'intero intervallo utile di probabilita' e' coperto da 2–3 livelli. La griglia esplorata ($K \in \{10, 12, 14\}$) e' probabilmente troppo grossolana: valori di K sensibilmente maggiori (32, 64) meritano di essere testati.

2. **Prior uniforme.** Con transizione uniforme e $K = 10$, la distribuzione limite assegna probabilita' $1/10$ a ciascun livello, ma la distribuzione empirica dei valori del graphon e' fortemente concentrata sui livelli bassi. Il modello deve quindi spendere gran parte della traiettoria inversa per correggere la prior. La transizione *marginale* di DiGress e' progettata esattamente per questo caso ed e' gia' predisposta nel codice.

3. **Numero di graphon indipendenti.** Anche con $L = 800$, i graphon provengono da $C = 50$ cluster: la diversita' effettiva e' limitata dal numero di cluster piu' che dal numero di repliche.

4. **Doppia quantizzazione.** Il formato di scambio PNG a 8 bit (Osservazione 3.2) introduce una perdita di informazione evitabile, sostituibile con la serializzazione diretta dei tensori.

5. **Modellazione ridondante in D3PM.** Modellare tutte le 16 384 celle anziche' le 8 128 del triangolo superiore raddoppia la dimensionalita' del problema senza aggiungere informazione.

Capitolo 5

Conclusioni provvisorie

La pipeline OGAD propone una scomposizione modulare del problema di generazione di grafi: prima si stimano graphon dai dati osservati, poi si apprende una distribuzione sui graphon, infine si campionano grafi e li si valuta con metriche strutturali. Questa impostazione rende esplicita la rappresentazione intermedia appresa dal modello e permette di confrontare generatori discreti diversi nello stesso protocollo sperimentale, come mostrato dalle due varianti D3PM e DiGress che condividono lo stesso schema di quantizzazione, lo stesso stadio di campionamento e lo stesso protocollo di valutazione.

I risultati preliminari indicano che la rappresentazione a graphon, nella forma attualmente implementata, non e' ancora competitiva con la generazione diretta di grafi. L'analisi dettagliata dell'implementazione suggerisce pero' che le cause piu' probabili siano scelte di discretizzazione e di prior, entrambe correggibili senza modifiche architetturali.

Il programma di lavoro che ne deriva e' il seguente:

- **Diagnostica del collo di bottiglia**: misurare la MMD dei grafi campionati direttamente dai graphon stimati da SIGL, per separare l'errore di stima dall'errore di generazione;
- **Ablation sulla quantizzazione**: estendere la griglia di K a valori significativamente maggiori, verificando il trade-off fra risoluzione di probabilita' e difficolta' del problema categoriale;
- **Ablation sulla transizione**: confrontare transizione uniforme e marginale nella variante DiGress, dove la seconda e' gia' implementata;
- **Confronto equo fra le due varianti**: eseguire OGAD–D3PM e OGAD–DiGress sulla stessa configurazione SIGL, con lo stesso K, lo stesso numero di graphon generati e lo stesso numero di grafi valutati;

- **Consolidamento del protocollo**: uniformare la cardinalita' dei tre insiemi confrontati, aggiungere la MMD training–test come scala di riferimento secondo la raccomandazione di GGME, ed estendere la valutazione alle perturbazioni diverse da `AddEdges`;
- **Semplificazioni implementative**: eliminare il formato di scambio PNG e modellare il solo triangolo superiore nella variante D3PM.

Bibliografia

[1] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in Neural Information Processing Systems*, 34:17981–17993, 2021.

[2] Ali Azizpour, Nicolas Zilberstein, and Santiago Segarra. Scalable implicit graphon learning. In *International Conference on Artificial Intelligence and Statistics*, pages 3952–3960, 2025.

[3] Christian Borgs and Jennifer Chayes. Graphons: A nonparametric method to model, estimate, and design algorithms for massive networks. Microsoft Research preprint, 2017.

[4] Stanley H. Chan and Edoardo M. Airoldi. A consistent histogram estimator for exchangeable graph models. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 208–216. PMLR, 2014.

[5] Nicola De Cao and Thomas Kipf. Molgan: An implicit generative model for small molecular graphs, 2018.

[6] Persi Diaconis and Svante Janson. Graph limits and exchangeable random graphs. *Rendiconti di Matematica e delle sue Applicazioni*, 28(1):33–61, 2008.

[7] Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. In *AAAI Workshop on Deep Learning on Graphs: Methods and Applications*, 2021.

[8] Chao Gao, Yu Lu, and Harrison H. Zhou. Rate-optimal graphon estimation. *The Annals of Statistics*, 43(6):2624–2652, 2015.

[9] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773, 2012.

[10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

[11] Emiel Hoogeboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. Argmax flows and multinomial diffusion: Learning categorical distributions. In *Advances in Neural Information Processing Systems*, volume 34, pages 12454–12465, 2021.

[12] Laszlo Lovasz. *Large Networks and Graph Limits*, volume 60 of *American Mathematical Society Colloquium Publications*. American Mathematical Society, 2012.

[13] Laszlo Lovasz and Balazs Szegedy. Limits of dense graph sequences. *Journal of Combinatorial Theory, Series B*, 96(6):933–957, 2006.

[14] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8162–8171. PMLR, 2021.

[15] Leslie O'Bray, Max Horn, Bastian Rieck, and Karsten Borgwardt. Evaluation metrics for graph generative models: Problems, pitfalls, and practical solutions. In *International Conference on Learning Representations*, 2022.

[16] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, 2023.

[17] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[18] Simo Ryu. Minimal implementation of a d3pm (structured denoising diffusion models in discrete state-spaces), in pytorch. `https://github.com/cloneofsimo/d3pm`, 2024. Repository used as implementation reference.

[19] Martin Simonovsky and Nikos Komodakis. Graphvae: Towards generation of small graphs using variational autoencoders. In *Artificial Neural Networks and Machine Learning – ICANN 2018*, pages 412–422. Springer, 2018.

[20] Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, pages 7462–7473, 2020.

[21] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 2256–2265. PMLR, 2015.

[22] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

[23] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[24] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, et al. Llama: Open and efficient foundation language models, 2023.

[25] Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. Digress: Discrete denoising diffusion for graph generation. In *The Eleventh International Conference on Learning Representations*, 2023.

[26] Xinyue Xia, Gal Mishne, and Yusu Wang. Implicit graphon neural representation. In *International Conference on Artificial Intelligence and Statistics*, pages 10619–10634, 2023.

[27] Jiaxuan You, Rex Ying, Xiang Ren, William Hamilton, and Jure Leskovec. Graphrnn: Generating realistic graphs with deep auto-regressive models. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 5708–5717. PMLR, 2018.